Implementing Threads

CSE 493L

Implementing Threads

- Threads are cool
- How do we implement them on one core?
- How do we implement coroutines running on multiple cores?
- How do we implement threads running on multiple cores?

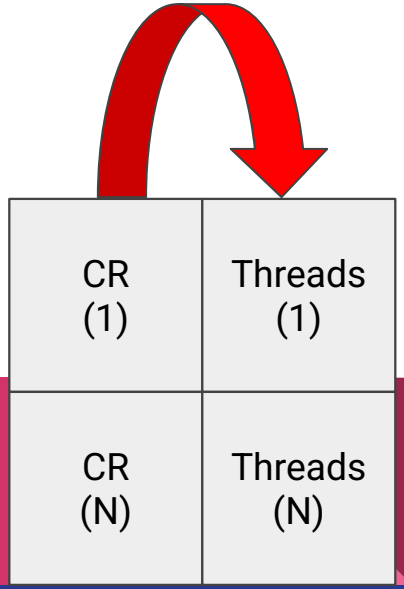
Agenda

- Starting from homework 1's singlecore coroutine implementation,
- From cooperative routines to uncooperative (threads)
- From singlecore execution to multicore coroutine execution
- Combining them: multicore threading!

		Preemptive?	
		No	Yes
Multi-core?	No	CR (1)	Threads (1)
	Yes	CR (N)	Threads (N)

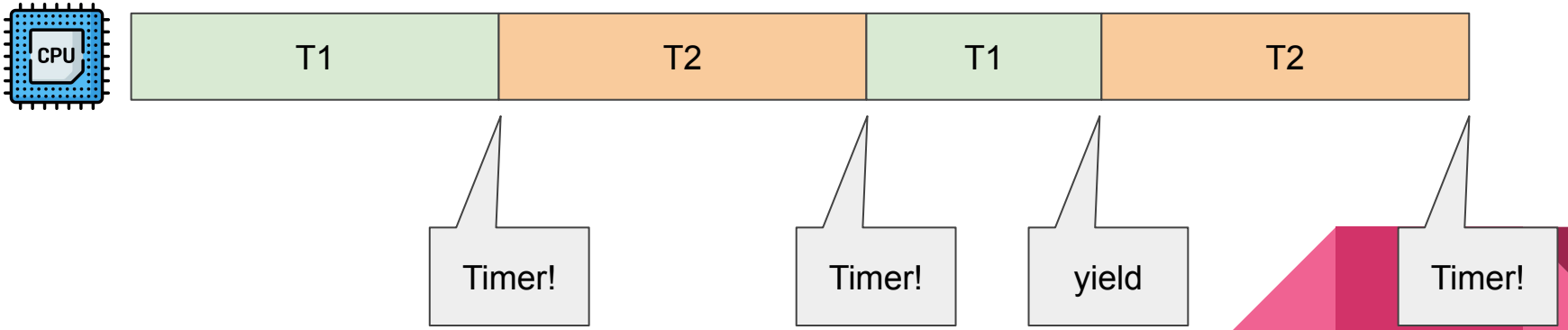
Let’s implement threads, starting from our HW 1 code

- For now, assume just one core running coroutines
- We have no synchronization primitives available to us
  - We are building the synchronization primitives!



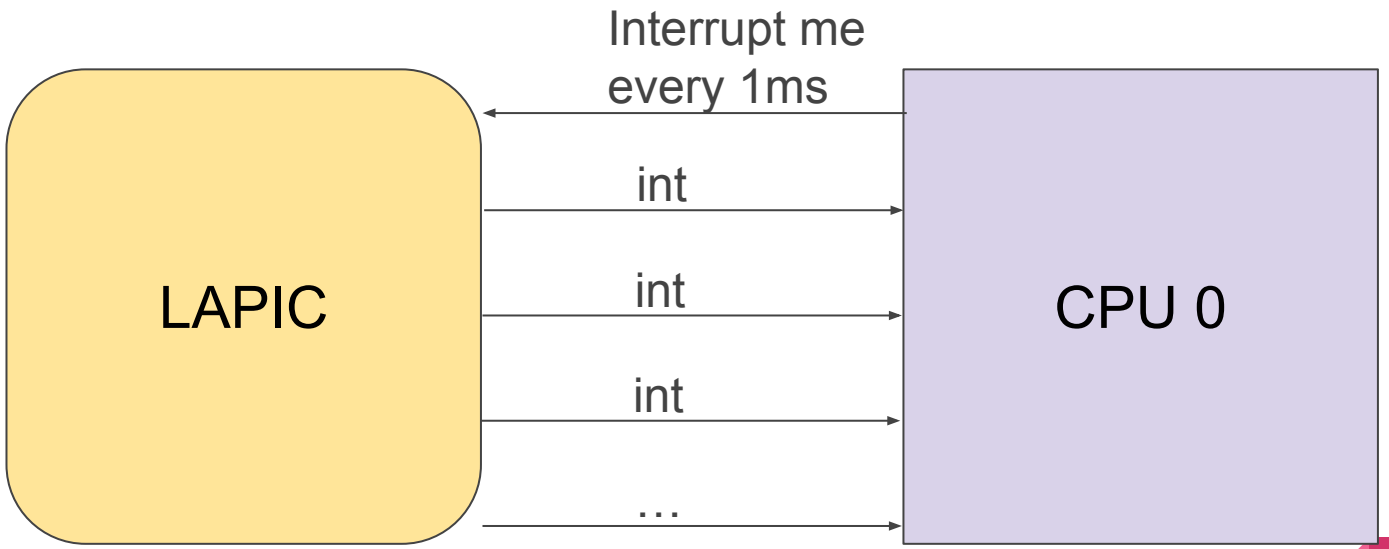
Threads

- We want to be robust to coroutines that “forget” to insert yield()
- We need threads: routines that are forced to yield every so often!
  - How? Hardware!



The LAPIC

(core-)Local Advanced Programmable Interrupt Controller



x86 Timer API

We have 2 assembly instructions: **`sti`** and **`cli`**

- **`cli`**
  - Disable/mask interrupts
- **`sti`**
  - Enable/unmask interrupts

What does the CPU do when an interrupt occurs?

The hardware does something according to the manual!

x86: finish current instruction, backup regs, call function at IDT[0]

IDT = Interrupt Descriptor Table, exists at a known memory location

What does the CPU do when an interrupt occurs?

In practice:

```
void on_timer_interrupt() {
    printf("timer interrupt occurred!\n");
}
```

What does the CPU do when an interrupt occurs?

In practice:

```
void on_timer_interrupt() {

    printf("timer interrupt occurred!\n");

}
```

Returns back to interrupted task

What does the CPU do when an interrupt occurs?

In practice:

```
void on_timer_interrupt() {
    printf("timer interrupt occurred!\n");
}
```

What if this itself takes longer than 1ms?

What does the CPU do when an interrupt occurs?

In practice:

```
void on_timer_interrupt() {
    printf("timer interrupt occurred!\n");
}
```

What if this itself takes longer than 1ms?

The CPU also disables interrupts in the interrupt handler!

Great! Now we can type:

```
void on_timer_interrupt() {
    yield();
}
```

Our homework 1 yield
is not thread-safe :(

The pitfall

Our hw1 yield() function has data races when we enable interrupts!

- e.g. shared mutable state without synchronized accesses

- Wait, aren’t we only running on one core? How can we have a data race?

Homework 1 Yield

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

What if an interrupt occurs here?

Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

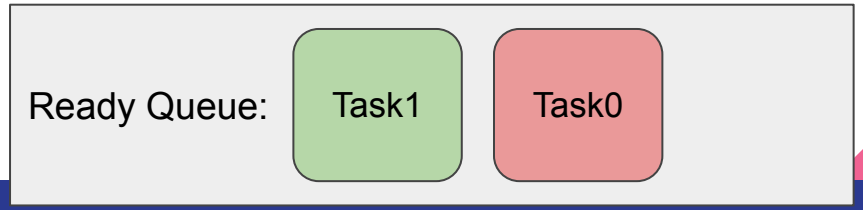
Task 0 calls yield

Ready Queue: Task1

Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

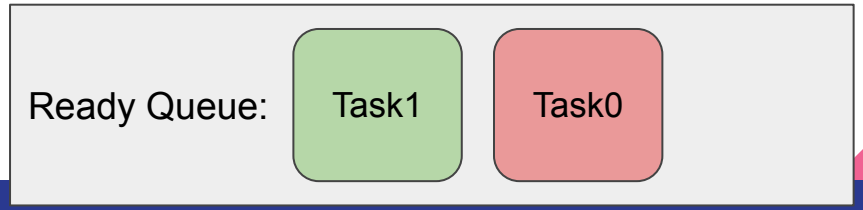
Task 0 on RQ,
registers haven't yet been
snapshotted



Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

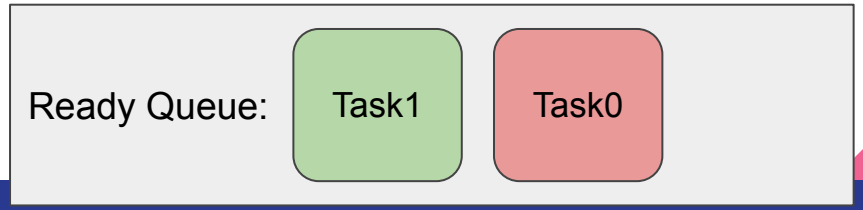
Interrupt!



Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

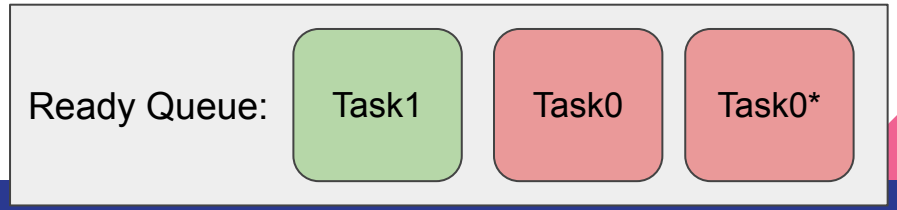
interrupt handler calls yield



Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

We now push Task0 again!



Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

next = Task1

Ready Queue:

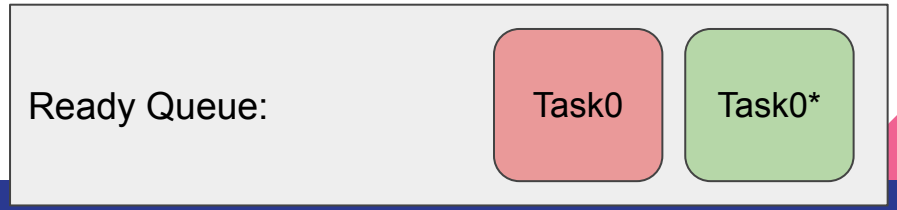
Task0

Task0*

Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

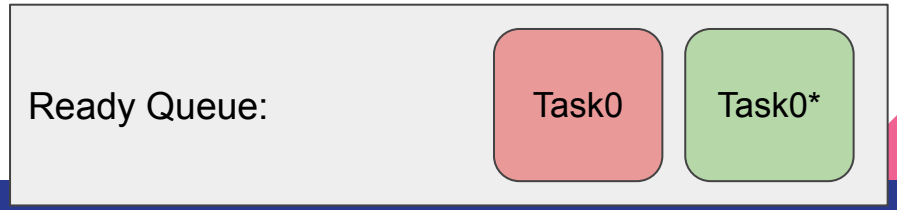
Task0* registers saved,
Task0 remains corrupt



Where did we go wrong?

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

Next call to yield() will run a corrupted Task0!



Where did we go wrong?

```
void yield() {
    disable_interrupts();
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    enable_interrupts();
}
```

Where did we go wrong?

```
void yield() {
    disable_interrupts();
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    enable_interrupts();
}
```

Critical Section

Critical Section

- A chunk of code that only 1 thread can be running at a time
- With only 1 core, a critical section can be implemented via interrupt masking!

Real world considerations

- We should only enable interrupts if they were enabled before the call to yield
- What if you forget to call enable_interrupts() after disabling?
  - C++: You can have an “Interrupt Guard”: a class who’s constructor disables interrupts and who’s destructor reenables them if they were enabled before
  - You no longer have to remember to manually reenable interrupts

Why did we have this bug?

- Old school of thought
  - We did not place a critical section around code that is not thread-safe
  - We need to check for code chunks that are critical w.r.t. other code chunks
- New school of thought
  - We had shared mutable data (the ready queue) without synchronization
  - We just need to check for all shared mutable references (e.g. globals)
    - “Shared Mutable References” => when two threads have a pointer and are mutating the data stored behind the pointer, e.g. a data race

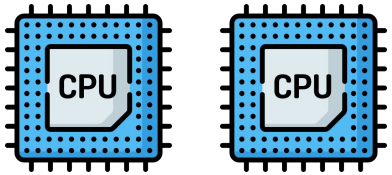
Threads

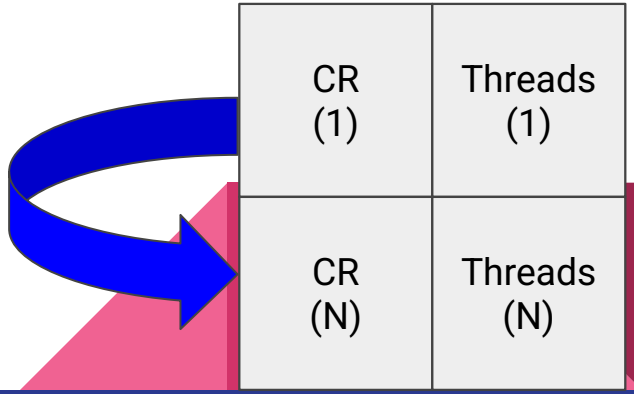
We have graduated from coroutines to threads!

- We upgraded our routines with timers that call yield
- We had to rewrite our entire coroutine library to be single-core thread safe
  - We need critical sections to protect global variables like the ready queue
  - We will need more advanced concepts to implement multicore critical sections with interrupts
- If we don’t have access to a LAPIC…
  - The OS can deliver interrupts to your process, called “signals”
  - You can mask and unmask signals just like interrupts
  - This is how homework 3 will work, under the hood

Moving on to Multi core

We’re gonna turn off the timer for now, and just try and make coroutines run in multi core





Let’s run homework 1 (coroutines) on 2 cores!

What could go wrong?

Let’s run homework 1 (coroutines) on 2 cores!

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

Let’s run homework 1 (coroutines) on 2 cores!

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

Both cores share the same ready_queue.
What if they both call yield() at the same time?

Let’s run homework 1 (coroutines) on 2 cores!

```
void yield() {
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
}
```

What if core 0 is here, and then core 1 pops `current` and attempts to switch to it before core 0 switches away?

Let’s run homework 1 (coroutines) on 2 cores!

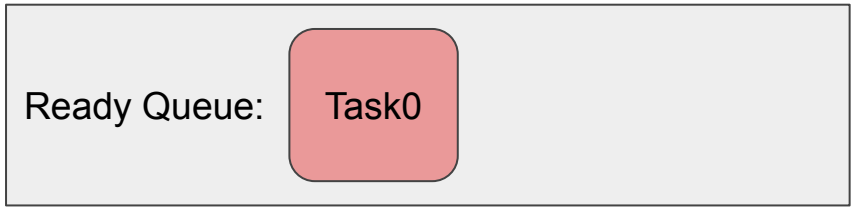
CPU 0:

CPU 1:

Ready Queue:

Let’s run homework 1 (coroutines) on 2 cores!

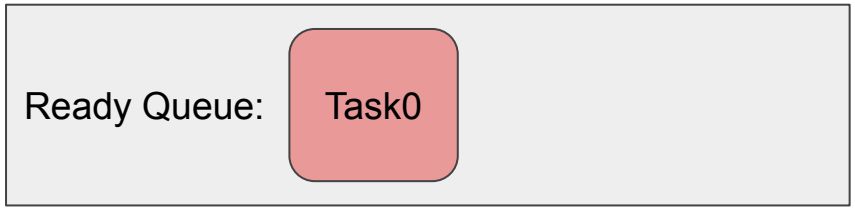
```
CPU 0:                                          CPU 1:

ready_queue.push(current);
```



Let’s run homework 1 (coroutines) on 2 cores!

```
CPU 0:

ready_queue.push(current);

next = ready_queue.pop();
```

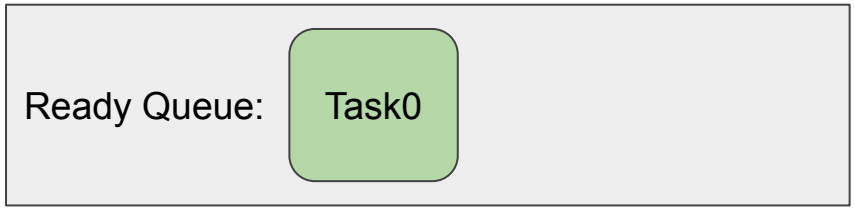
```
CPU 1:
```



Let’s run homework 1 (coroutines) on 2 cores!

```
CPU 0:

ready_queue.push(current);

next = ready_queue.pop();

switch(next, current);
```

```
CPU 1:
```



Let’s run homework 1 (coroutines) on 2 cores!

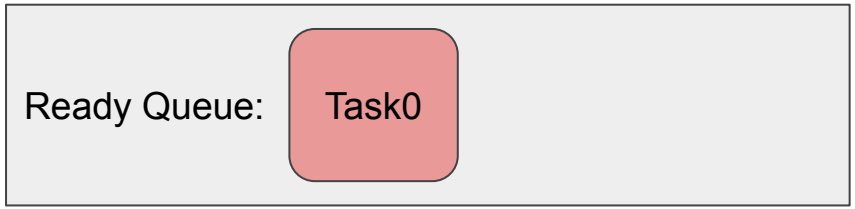
CPU 0:

CPU 1:

Ready Queue:

Let’s run homework 1 (coroutines) on 2 cores!

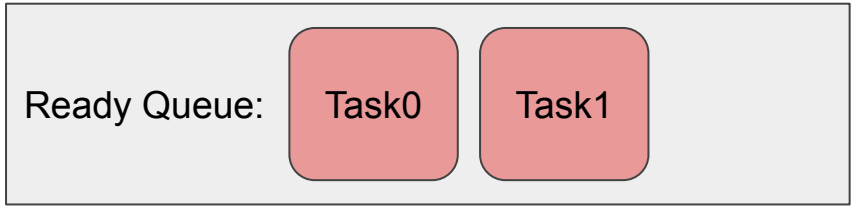
```
CPU 0:                                          CPU 1:

ready_queue.push(current);
```



Let’s run homework 1 (coroutines) on 2 cores!

```
CPU 0:

ready_queue.push(current);
```

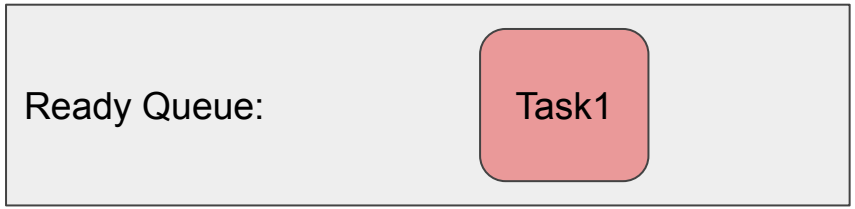
```
CPU 1:

ready_queue.push(current);
```



Let’s run homework 1 (coroutines) on 2 cores!

```
CPU 0:

ready_queue.push(current);
```

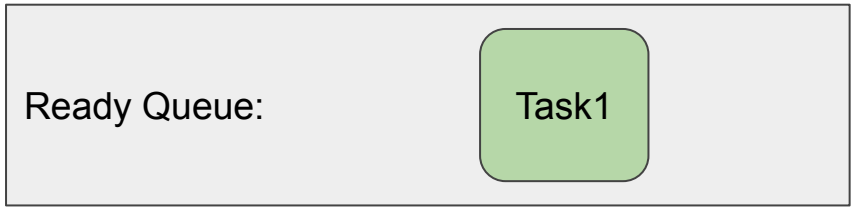
```
CPU 1:

ready_queue.push(current);

next = ready_queue.pop();
```



Let’s run homework 1 (coroutines) on 2 cores!

```
CPU 0:

ready_queue.push(current);
```

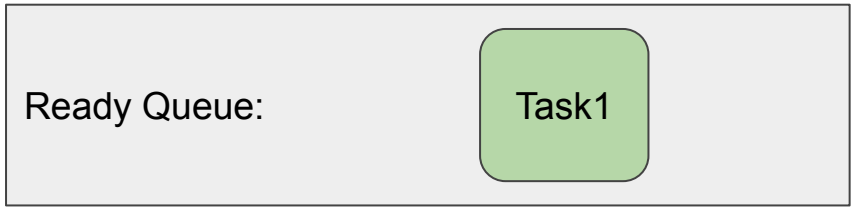
```
CPU 1:

ready_queue.push(current);

next = ready_queue.pop();

switch(next, current);
```



Let’s run homework 1 (coroutines) on 2 cores!

```
CPU 0:

ready_queue.push(current);

next = ready_queue.pop();

switch(next, current);
```

```
CPU 1:

ready_queue.push(current);

next = ready_queue.pop();

switch(next, current);
```



We still need a critical section here…

```
void yield() {
    critical_begin();
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    critical_end();
}
```

We still need a critical section here…

```
void yield() {
    critical_begin();
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    critical_end();
}
```

This time, the critical section needs to protect against parallel running of the same code!

Secret Synchronization Primitives

Synchronization primitives come in two flavors:

- Blocking (go onto a queue until you can run)
  - Not helpful since we're implementing yield itself
- Spinning (loop until you can run)
  - Helpful!

SpinLock

Spinning variant of Mutex

Instead of blocking when we can’t acquire the lock, we just keep on checking!

Let’s implement a SpinLock.

SpinLocked Yield, Broken:

```
bool lock = false;
void yield() {
    while (lock) {} // wait until lock is false
    lock = true;
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    lock = false
}
```

Compiler Attack!

The compiler places **`lock`** in a register, and realizes that constantly rereading a register will always return the same thing

```
bool lock = false;
void yield() {
    while (lock) {} // wait until lock is false
    lock = true;
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    lock = false
}
```

Compiler Attack!

The compiler places **`lock`** in a register, and realizes that constantly rereading a register will always return the same thing

```
bool lock = false;

void yield() {

    if (lock) {while (1) {}} // uh oh

    lock = true;

    ready_queue.push(current);

    next = ready_queue.pop();

    switch(next, current);

    lock = false

}
```

volatile keyword

Keep reading from memory, there are side effects, do not optimize!

```
volatile bool lock = false;
void yield() {
    while (lock) {} // wait until lock is false
    lock = true;
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    lock = false
}
```

This is how it’s supposed to go

lock = false

CPU 0:

CPU 1:

This is how it’s supposed to go

lock = false

CPU 0:

```
while (lock) {}
```

CPU 1:

This is how it’s supposed to go

lock = true

CPU 0:

```
while (lock) {}

lock = true;
```

CPU 1:

This is how it’s supposed to go

lock = true

CPU 0:

```
while (lock) {}

lock = true;

ready_queue.push(current);
```

CPU 1:

This is how it’s supposed to go

lock = true

CPU 0:

```
while (lock) {}

lock = true;

ready_queue.push(current);
```

CPU 1:

```
while (lock) {}
```

This is how it’s supposed to go

lock = true

CPU 0:

```
while (lock) {}

lock = true;

ready_queue.push(current);

next = ready_queue.pop();

switch(next, current);
```

CPU 1:

```
while (lock) {}
```

This is how it’s supposed to go

lock = false

CPU 0:

```
while (lock) {}

lock = true;

ready_queue.push(current);

next = ready_queue.pop();

switch(next, current);

lock = false
```

CPU 1:

```
while (lock) {}
```

This is how it’s supposed to go

lock = true

CPU 0:

```
while (lock) {}
lock = true;
ready_queue.push(current);
next = ready_queue.pop();
switch(next, current);
lock = false
```

CPU 1:

```
while (lock) {}
lock = true;
```

This is how it’s supposed to go

lock = true

CPU 0:

```
while (lock) {}
lock = true;
ready_queue.push(current);
next = ready_queue.pop();
switch(next, current);
lock = false
```

CPU 1:

```
while (lock) {}
lock = true;
ready_queue.push(current);
next = ready_queue.pop();
switch(next, current);
```

This is how it’s supposed to go

lock = false

CPU 0:

```
while (lock) {}
lock = true;
ready_queue.push(current);
next = ready_queue.pop();
switch(next, current);
lock = false
```

CPU 1:

```
while (lock) {}
lock = true;
ready_queue.push(current);
next = ready_queue.pop();
switch(next, current);
lock = false
```

This can go wrong:

```
volatile bool lock = false;
void yield() {
    while (lock) {} // wait until lock is false
    lock = true;
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    lock = false
}
```

This can go wrong:

```
volatile bool lock = false;
void yield() {
    while (lock) {} // wait until lock is false
    lock = true;
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    lock = false
}
```

Two cores can both “think” they have the lock!

This is bad

lock = false

CPU 0:

```
while (lock) {}
```

CPU 1:

This is bad

lock = false

CPU 0:

```
while (lock) {}
```

CPU 1:

```
while (lock) {}
```

This is bad

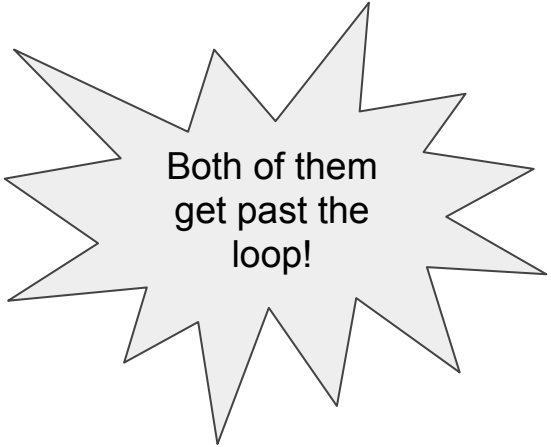
lock = false

CPU 0:

```
while (lock) {}
```

CPU 1:

```
while (lock) {}
```



This is bad

lock = true

CPU 0:

```
while (lock) {}

lock = true;
```

CPU 1:

```
while (lock) {}
```

This is bad

lock = true

CPU 0:

```
while (lock) {}

lock = true;
```

CPU 1:

```
while (lock) {}

lock = true;
```

This is bad

lock = true

CPU 0:

```
while (lock) {}

lock = true;

ready_queue.push(current);
```

CPU 1:

```
while (lock) {}

lock = true;
```

This is bad

lock = false

CPU 0:

```
while (lock) {}

lock = true;

ready_queue.push(current);
```

CPU 1:

```
while (lock) {}

lock = true;

ready_queue.push(current);

next = ready_queue.pop();

switch(next, current);

lock = false
```

This is bad

lock = false

CPU 0:

```
while (lock) {}
lock = true;
ready_queue.push(current);
next = ready_queue.pop();
switch(next, current);
lock = false
```

CPU 1:

```
while (lock) {}
lock = true;
ready_queue.push(current);
next = ready_queue.pop();
switch(next, current);
lock = false
```

What went wrong?

```
volatile bool lock = false;
void yield() {
    while (lock) {} // wait until lock is false
    lock = true;
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    lock = false
}
```

Time of check to time of use (ToCToU) bug

What went wrong?

```
volatile bool lock = false;
void yield() {
    while (lock) {} // wait until lock is false
    lock = true;
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    lock = false
}
```

We need hardware support to implement multithreaded mutual exclusion!

Atomics

Atomics

```
atomic bool lock = false;

void yield() {

   while (test_and_set(lock)) {} // atomically wait until lock is false

   ready_queue.push(current);

   next = ready_queue.pop();

   switch(next, current);

   lock = false

}
```

Atomics

```
atomic bool lock = false;

void yield() {

   while (test_and_set(lock)) {} // atomically wait until lock is false

   ready_queue.push(current);

   next = ready_queue.pop();

   switch(next, current);

   lock = false

}
```

Set lock to true, return old value

Atomics

```
atomic bool lock = false;

void yield() {

   while (test_and_set(lock)) {} // atomically wait until lock is false

   ready_queue.push(current);

   next = ready_queue.pop();

   switch(next, current);

   lock = false

}
```

Now, we only get past this loop once *we* were the core to change lock from False to True

Atomics

```
atomic bool lock = false;

void yield() {

    while (test_and_set(lock)) {} // atomically wait until lock is false

    ready_queue.push(current);

    next = ready_queue.pop();

    switch(next, current);

    lock = false

}
```

The hardware knows to not let ANY other core modify **`lock`** during an atomic operation!
(It does this by not replying to CC requests briefly.)

Atomic primitives (they're all the same)

We only need **one** of these to implement the rest:

`test_and_set`: atomically set the value AND get the old value

`compare_and_swap`: atomically set the value IF old value is x

`fetch_add`: atomically increment the value (avoiding the RMW race condition)

class SpinLock:

```
atomic<bool> atom;

void lock() {

    while (atom.test_and_set(true)) {}

}

void unlock() {

    atom.set(false)

}
```

class SpinLock:

```
atomic<bool> atom;

void lock() {

    while (atom.test_and_set(true)) {}

}

void unlock() {

    atom.set(false)

}
```

This wastes a lot of energy!
The core is just spinning!

class SpinLock:

```
atomic<bool> atom;

void lock() {

   while (atom.test_and_set(true)) {pause();}

}

void unlock() {

   atom.set(false)

}
```

Tell the CPU to slow down

class SpinLock:

```
atomic<bool> atom;

void lock() {

    while (atom.test_and_set(true)) {monitor_mwait(atom);}

}

void unlock() {

    atom.set(false)

}
```

Tell the CPU to turn off until atom is changed.
Only available to kernel code

class SpinLock:

```
atomic<bool> atom;

void lock() {

    while (atom.test_and_set(true)) {}

}

void unlock() {

    atom.set(false)

}
```

Takeaway: SpinLocks should only be held for a short duration!

class SpinLock:

```
atomic<bool> atom;

void lock() {

    while (atom.test_and_set(true)) {yield();}

}

void unlock() {

    atom.set(false)

}
```

Question: why can't we call yield() here?

Non-preemptive Contexts

contexts where yielding is not allowed

e.g., within yield()

These are the times we use SpinLocks; we would love to yield() if we could!

SpinLock things to not do

Wouldn’t it be nice if the compiler helped us with these rules?

- Forget to call unlock()
- Call yield while locked
- Call lock() twice
- Create a cyclic dependency
  - Thread 1 locks lock A, then B
  - Thread 2 locks lock B, then A
  - Both succeed their first lock, spin forever on their second lock
  - “Deadlock”

Don’t yield while holding a SpinLock

CR 1	CR 2
lock.lock() yield()	lock.lock() …..

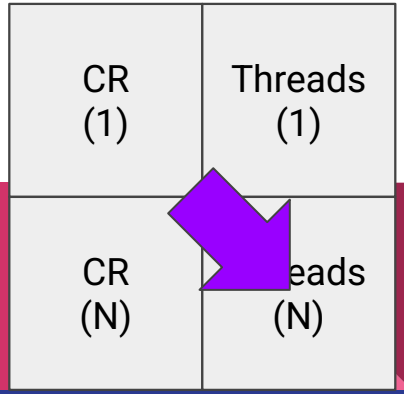
If we yield after a lock.lock(), and then we run into another lock.lock(), we might spin forever!

Not always a correctness bug, but in practice, catastrophic for performance

If we tried to re-introduce preemption…

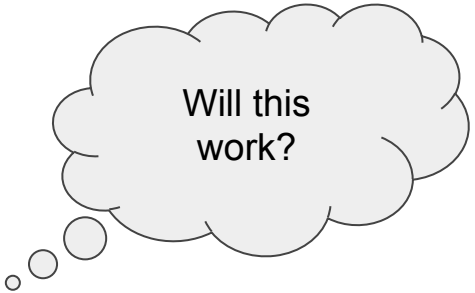
We might preemptibvely yield while a SpinLock is held!

We need a way to SpinLock without interrupts enabled



Interrupt Safe SpinLock (ISL):

```
atomic<bool> atom;
void lock() {
    while (atom.test_and_set(true)) {
        pause();
    }
    disable_interrupts();
}
void unlock() {atom.set(false); enable_interrupts();}
```



Interrupt Safe SpinLock (ISL):

```
atomic<bool> atom;
void lock() {
    while (atom.test_and_set(true)) {
        pause();
    }
    disable_interrupts();
}
void unlock() {atom.set(false); enable_interrupts();}
```

Could get interrupted here while holding the lock!

Interrupt Safe SpinLock (ISL):

```
atomic<bool> atom;
void lock() {
    disable_interrupts();
    while (atom.test_and_set(true)) {
        enable_interrupts();
        pause();
        disable_interrupts();
    }
}
void unlock() {atom.set(false); enable_interrupts();}
```

Upon returning, interrupts are turned off

Interrupt Safe SpinLock (ISL):

```
atomic<bool> atom;
void lock() {
    disable_interrupts();
    while (atom.test_and_set(true)) {
        enable_interrupts();
        pause();
        disable_interrupts();
    }
}
void unlock() {atom.set(false); enable_interrupts();}
```

In practice, restore them to old state rather than unconditionally enabling

We now have all the primitives required

to implement multicore, interrupt-enabled yield()!

critical_begin() = isl.lock()

critical_end() = isl.unlock()

thread-safe yield implementation

```
void yield() {
    isl.lock();
    ready_queue.push(current);
    next = ready_queue.pop();
    switch(next, current);
    isl.unlock();
}
```

Lessons learned

- Shared mutable state (e.g. the ReadyQueue) is the root of all evil
  - Requires synchronization between threads!
- Locks are hard
- Interrupts are hard
- Multicore is harder
- Multicore with interrupts is extra hard
  - Every instruction can be interleaved and/or interrupted unless in a critical section
  - Spinlocks are almost never used, ISLs are *the* primitive

Lessons learned

- SpinLocks, ISLs are useful for protecting your scheduler’s code
- SpinLocks are dangerous to use in User Code!
  - Getting preempted while holding a spinlock is not good!
- ISLs are typically not exposed to User Code
- Mutex is what User Code wants